

Exercise 1 – Scalar Function (Basic)

Query Query History

```
1 CREATE FUNCTION get_customer_fullname (p_customer_id INT)
2 RETURNS TEXT
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6 v_full_name TEXT;
7 BEGIN
8 SELECT first_name || ' ' || last_name
9 INTO v_full_name
10 FROM customer
11 WHERE p_customer_id = customer_id;
12 RETURN v_full_name;
13 END;
14 $$;
15 SELECT get_customer_fullname(1)
16
```

Data Output Messages Notifications

SQL

S

	get_customer_fullname text
1	Mary Smith

Exercise 2 – Table-Valued Function

Query Query History

```
1 CREATE FUNCTION get_films_by_category (p_category_name TEXT)
2 RETURNS TABLE (
3 film_id INT,
4 title VARCHAR,
5 release_year YEAR,
6 rental_rate NUMERIC
7 )
8 LANGUAGE plpgsql
9 AS $$
10 DECLARE
11 v_full_name TEXT;
12 BEGIN
13 RETURN QUERY
14 SELECT f.film_id,
15 f.title,
16 f.release_year,
17 f.rental_rate
18 FROM film f
19 JOIN film_category fc
20 ON fc.film_id = f.film_id
21 JOIN category c
22 ON fc.category_id = c.category_id
23 WHERE p_category_name = c.name;
24 END;
25 $$;
26 SELECT * FROM get_films_by_category('Action')
27
```

Data Output Messages Notifications

SQL

Showing r

film_id	title	release_year	rental_rate
---------	-------	--------------	-------------

Exercise 4 – Data Processing Function

Query Query History

```
1 CREATE FUNCTION get_customer_total_payment(p_customer_id INT)
2 RETURNS NUMERIC
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6 v_total_payment NUMERIC;
7 BEGIN
8 SELECT COALESCE(SUM(amount))
9 INTO
10 v_total_payment
11 FROM payment
12 WHERE p_customer_id = customer_id;
13 RETURN v_total_payment;
14 END;
15 $$;
16 SELECT get_customer_total_payment(1)
17
18 -- DROP FUNCTION IF EXISTS get_customer_fullname(INT);
19 -- RESET ALL;
```

Data Output Messages Notifications



	get_customer_total_payment	
	numeric	
1	114.70	

Exercise 5 – Advanced Function

```
CREATE FUNCTION get_top_customers(p_limit INT)
RETURNS TABLE (
customer_id INT,
full_name TEXT,
total_payment NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
RETURN QUERY
SELECT
c.customer_id,
c.first_name || ' ' || c.last_name AS full_name,
sum(p.amount) AS total_payment
FROM customer c
JOIN payment p
ON p.customer_id = c.customer_id
GROUP BY
c.customer_id,
c.first_name,
c.last_name
ORDER BY SUM(p.amount) DESC
LIMIT p_limit;
END;
$$;
SELECT * FROM get_top_customers(5);
```

									SQL
	customer_id integer					full_name text		total_payment numeric	
1	148					Eleanor Hunt		211.55	
2	526					Karl Seal		208.58	
3	178					Marion Snyder		194.61	
4	137					Rhonda Kenne...		191.62	
5	144					Clara Shaw		189.60	